Asian Journal of Advances in Agricultural Research

Volume 25, Issue 10, Page 53-67, 2025; Article no.AJAAR.144906
ISSN: 2456-8864

Smart Prediction of Shelf Life and Tomato Sorting Using Deep Learning

G.Geetha [a++*], P.C. Prabhu Kumar [b#], V.Suriyaraj [a] and J.Naveen Kumar [b]

[a] Department of Information Science and Technology, College of Engineering, Anna University, Chennai, Tamilnadu, India.
[b] Department of Computer Science and Engineering, Mother Theresa Institute of Engineering and Technology, Palamaneru, Andharapradesh, India.

Authors' contributions

This work was carried out in collaboration among all authors. All authors read and approved the final manuscript.

Article Information

DOI: https://doi.org/10.9734/ajaar/2025/v25i10680

Open Peer Review History:
This journal follows the Advanced Open Peer Review policy. Identity of the Reviewers, Editor(s) and additional Reviewers, peer review comments, different versions of the manuscript, comments of the editors, etc are available here: https://pr.sdiarticle5.com/review-history/144906

Original Research Article

Received: 19/07/2025
Accepted: 27/09/2025
Published: 06/10/2025

ABSTRACT

Agriculture has been essential throughout history, sustaining communities and providing food worldwide. Over time, farming methods have evolved with new ideas and scientific advancements. Today, technology plays a crucial role in improving farming practices. Deep learning and computer vision are two such technologies that help farmers by providing smart solutions like crop monitoring and smart irrigation system. Tomatoes are a significant crop with versatile uses and high consumer demand. However, traditional methods of sorting tomatoes are labor-intensive, time consuming and prone to errors, leading to delays in the supply chain. To address this issue, the proposed system introduces a object detection model to enhance tomato sorting processes. In addition to software solutions, the proposed system aims to develop an automated sorting prototype equipped with

[++] https://orcid.org/0000-0003-4572-0258;
[#] https://orcid.org/0000-0002-3354-101X;
**Corresponding author: Email: ggeetha@annauniv.edu;*

Cite as: *G.Geetha, P.C. Prabhu Kumar, V.Suriyaraj, and J.Naveen Kumar. 2025. "Smart Prediction of Shelf Life and Tomato Sorting Using Deep Learning". Asian Journal of Advances in Agricultural Research 25 (10):53–67. https://doi.org/10.9734/ajaar/2025/v25i10680.*

specialized components and technology. This proposed system is capable of dynamically sorting tomatoes based on ripeness and quality. The proposed system carried out by using two models related to YOLOv8 types for determining the shelf life of tomato in smart way. In the present Food security era, this research delivers to sensitize the commercial tomato growers through sending some sensor base signals or warnings to concern people through IoT communication. As a tomato undergoes assessment by the computer, the prototype responds accordingly, minimizing manual intervention and reducing errors. By integrating sophisticated technology with practical machinery, our objective is to optimize agriculture methods. This proposed system aims to showcase the transformative potential of technology in agriculture by enabling efficient sorting of tomatoes, leading to increased productivity, resource efficiency, and food security.

Keywords: Deep learning; internet of things; machine learning; YOLO; CNN.

1. INTRODUCTION

In the domain of contemporary agriculture, technological advancements have played a pivotal role in fostering growth and innovation. From precision agriculture to automated irrigation systems, technology has reshaped the landscape of crop cultivation and agricultural management. However, despite these strides, the sorting and categorization of agricultural products remain significant challenges, particularly for highly sought-after commodities such as tomatoes. Tomatoes occupy a revered position in global cuisine, renowned for their versatility, vibrant colours, and nutritional value. Whether featured in salads, sauces, or sandwiches, their universal culinary appeal transcends cultural boundaries. Furthermore, the nutritional richness of tomatoes, characterized by their abundance of vitamins, minerals, and antioxidants, underscores their importance in promoting overall health and well-being.

The traditional sorting methods, deeply entrenched in agricultural traditions, often rely on manual labour and subjective judgment. While these methods may yield results to some extent, they are plagued by inefficiencies, particularly in terms of time consumption. Manual sorting necessitates extensive labour and time investment, as workers meticulously assess each tomato for attributes such as size, ripeness, and quality. This labour-intensive process not only prolongs sorting procedures but also heightens the risk of errors and inconsistencies. In response to these challenges, the proposed system presents a transformative approach to streamline tomato sorting processes. By harnessing state-of-the-art technology, including computer vision and advanced image processing techniques, the system seeks to automate and optimize tomato sorting. Through the integration of cutting-edge algorithms and servo motor systems, the proposed solution enables efficient and accurate tomato classification based on predefined criteria. By reducing the time-consuming burdens associated with manual sorting, this system has the potential to enhance efficiency, minimize waste, and ensure the consistent quality of tomatoes across the supply chain.

2. RELATED WORKS

2.1 Data Collection

Collecting data for object detection models aimed at identifying tomatoes involves assembling a diverse set of tomato images representing various types of tomatoes, ripeness levels, and lighting conditions. It's essential to gather a comprehensive collection of images to enable the model to learn to accurately recognize tomatoes in different contexts. High-quality images showcasing tomatoes from multiple perspectives are crucial for effective training, allowing the model to distinguish between tomatoes and other objects accurately. Additionally, manually annotating the tomatoes in these images with bounding boxes aids the model in understanding their appearance, akin to providing it with a map for learning. To ensure dataset diversity, images are sourced from different locations such as farms, greenhouses, and online databases, exposing the model to various environments where tomatoes may be encountered(Clement et al ,2012).

Keeping track of metadata, including tomato type and image location, helps in better understanding and evaluating the dataset. A Bayesian classifier with a multivariate and the three-class problem was implemented for data fusion (Baltazar et al., 2008). By meticulously collecting and preparing the data, the model can reliably detect and classify tomatoes in real-world scenarios, enabling its deployment in diverse settings such as farms or grocery stores where quick and accurate tomato identification is

essential. This meticulous data collection process ensures that the model is trained on a wide range of tomato images, enabling it to accurately identify tomatoes in various environments and lighting conditions. The annotated bounding boxes provide the model with precise information about the location and shape of tomatoes within the images, facilitating more accurate detection and classification. Additionally, the inclusion of metadata such as tomato type and image source allows for better understanding and evaluation of the dataset, ensuring that the model is trained on representative and diverse data. Ultimately, the careful collection and annotation of data contribute to the robustness and effectiveness of the object detection model in accurately identifying and classifying tomatoes in real-world scenarios, making it a valuable tool for applications such as agricultural monitoring and food quality inspection (Deng et al., 2009; Kuznetsova & Soloviev, 2020).

2.2 Data Preprocessing

Preprocessing methods are fundamental in optimizing input data for object detection models, significantly impacting their performance and accuracy. One crucial step is image resizing, which ensures that all input images conform to standardized dimensions. This uniformity is vital for efficient processing and consistent detection across diverse images, enabling the model to interpret visual data more effectively. Image augmentation techniques, such as rotation, flipping, scaling, and translation, play a pivotal role in enriching the training dataset. These methods create variations of the original images, allowing the model to learn from a broader range of scenarios. This enhances the model's ability to generalize and perform well on unseen data, reducing overfitting and improving robustness. Normalization of pixel values is another critical preprocessing step. By scaling pixel values to a specific range, usually between 0 and 1, normalization helps stabilize the training process. This stabilization leads to faster convergence during optimization, as the model's gradients become more manageable. Accurate data annotation with bounding boxes is essential for the supervised training of object detection models. These annotations provide the ground truth labels necessary for the model to learn to detect and localize objects within images effectively. Additionally, class balancing techniques are employed to prevent the model from becoming biased towards dominant classes, ensuring that the model learns equally across all object categories. Finally, quality assurance measures, including thorough data quality checks, are implemented to maintain the integrity of the training dataset. These steps collectively contribute to the effectiveness and robustness of object detection models, making preprocessing an indispensable component of the model development pipeline (Bishop, 2006; Bochkovskiy et al., 2020; Redmon et al., 2016).

2.3 Object Detection Model

Recent advancements in deep learning and image processing have greatly enhanced automated tomato sorting systems. The (Zhou et al., 2023) study explores the use of the Faster R-CNN model for detecting and classifying tomatoes based on their ripeness. The Faster R-CNN architecture is well-regarded for its strong object detection capabilities and was trained on a diverse dataset containing images of tomatoes at various ripeness stages. To make the model more robust against different lighting conditions and positions, the dataset was further augmented with synthetic transformations.

Key performance metrics such as mean Average Precision (mAP), intersection over union (IoU), and inference time were used to evaluate the model's effectiveness. The study's results demonstrated that the Faster R-CNN model achieved high detection accuracy and was capable of real-time processing. This makes it particularly suitable for use in automated sorting systems, where speed and accuracy are essential.YOLOv4, known for its real-time object detection capabilities, has been effectively applied to tomato classification based on ripeness stages. The model's architecture and the training process on a diverse dataset comprising ripe, half-ripe, and unripe tomatoes are explored. Through rigorous evaluation using precision, recall, and F1-score metrics, the effectiveness of YOLOv4 in tomato classification is demonstrated. This highlights the model's potential as a promising solution for automated sorting systems in agriculture, providing valuable insights into leveraging deep learning, specifically YOLOv4, to enhance efficiency and accuracy in tomato classification tasks (Gai & Yuan, 2023; Redmon & Farhadi, 2018; Girshick, 2015).

3. SYSTEM MODEL

3.1 System Architecture

The system design incorporates advanced computer vision and deep learning technologies

to automate tomato sorting with precision and efficiency. By harnessing IOT actuators. The proposed system translate classification results into real-time sorting actions, to overcome traditional sorting methods. Through seamless integration of cutting-edge technologies, our system streamlines agricultural processes and enhances overall productivity. Fig. 1 provides a brief overview of the proposed system.

3.2 Data Collection

For this project, data collection was conducted through a comprehensive approach, encompassing deeply captured images, web scraping, and leveraging publicly available datasets from platforms such as Kaggle and Roboflow. Among these sources, Roboflow emerged as a particularly valuable resource, providing a wide range of high-quality images for training our deep learning model. Roboflow's dataset comprised more than 10,000 instances of tomatoes, with 2,000 images encompassing six distinct classes and two sizes (normal and small). This extensive dataset allowed for thorough training and validation of the model, ensuring robust performance across various scenarios. By leveraging Roboflow's dataset alongside other sources, we were able to create a diverse and comprehensive dataset essential for training a reliable tomato sorting model.

3.3 Pre Processing

In preparation for training our deep learning model, a series of preprocessing steps were applied to the collected images. Data preprocessing is a crucial step in machine learning, particularly for image datasets. It prepares your images for training robust and reliable deep learning models. First, all images were resized to a uniform size of 416x416 pixels, a commonly used ratio for YOLO models. This resizing step ensures consistency in input dimensions across the dataset, facilitating efficient training and inference. Additionally, data augmentation techniques were employed to further enhance the diversity of the dataset and improve the model's generalization ability. Specifically, each image underwent augmentation with rotations of up to 15 degrees in both positive and negative directions. These rotations introduce variations in the orientation of the tomatoes, exposing the model to different perspectives and helping it become more robust to variations in real-world scenarios.

3.4 Model Building

YOLOv8 pretrained model is one of the most commonly used object detection models, known for its efficiency and accuracy. Unlike traditional methods that involved multiple steps, YOLOv8 processes images swiftly and accurately in a single step. When an image is inputted into a YOLOv8 pretrained model, it undergoes several layers of processing. In the backbone network, which acts as the foundation, convolutional layers and pooling layers analyze the key features of the image. These layers are responsible for recognizing essential details such

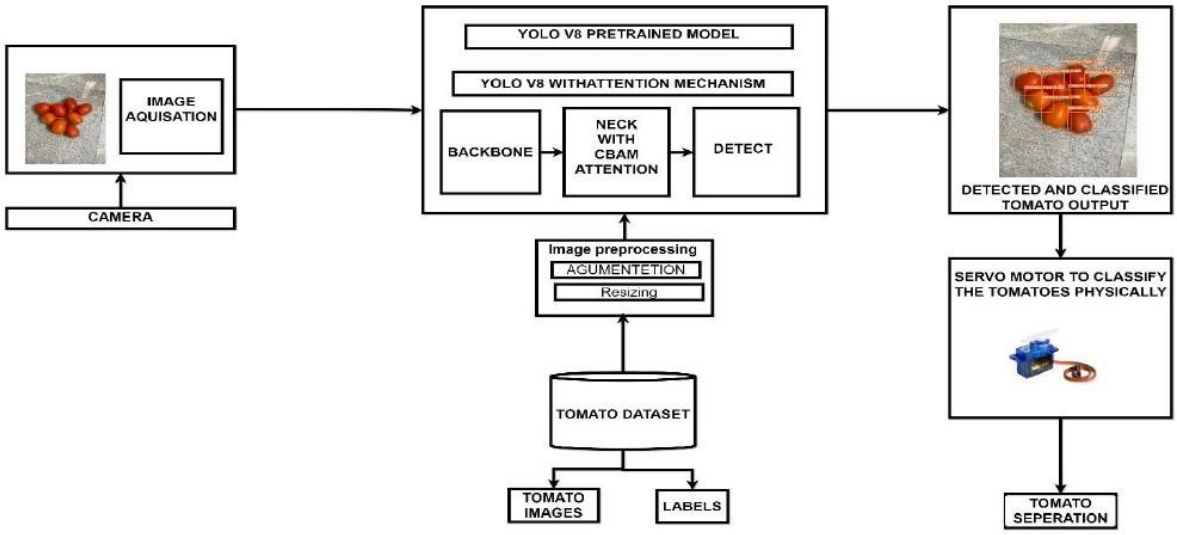

Fig. 1. Architecture of the proposed system

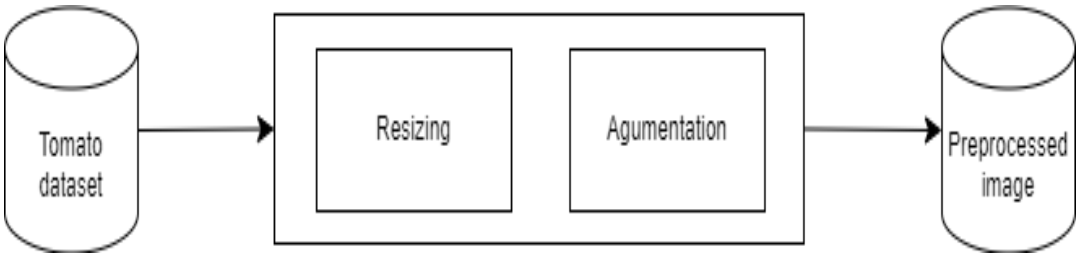

Fig. 2. Architecture of preprocessing module

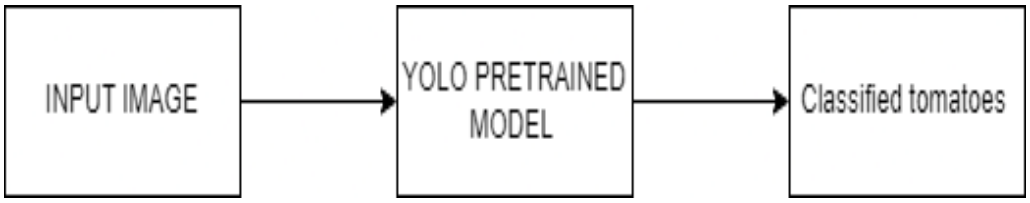

Fig. 3. Architecture of pretrained YOLO model

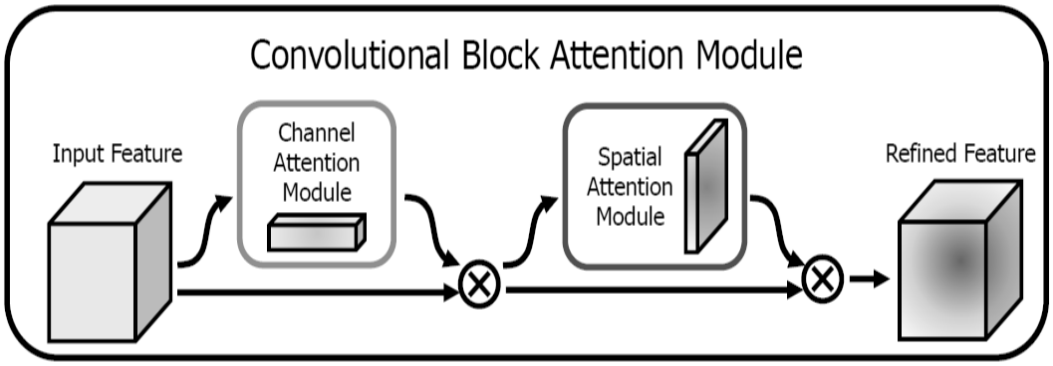

Fig. 4. Architecture of attention mechanism

as shapes, colors, and textures. Next, in the feature pyramid, the processed image is further analyzed to detect objects of various sizes. Layers within the feature pyramid create different scales of the image, enabling the model to identify objects more effectively across different scales and resolutions. Finally, in the detection head, layers are responsible for making predictions and drawing bounding boxes around detected objects. These layers classify the objects and determine their precise locations within the image (Wan et al., 2018; Zhao et al., 2023).

The ResBlock CBAM (Residual Block with Channel Attention Module) is an advanced architectural component in convolutional neural networks (CNNs) that integrates both channel attention and spatial attention mechanisms. By selectively emphasizing important channels and spatial regions while suppressing irrelevant ones, ResBlock CBAM enhances the model's ability to capture fine-grained details and subtle patterns in the input data.

YOLOv8 model with ResBlock attention mechanism, feature processing begins with those obtained from the backbone network, extracted at various stages. These features lay the groundwork for subsequent operations within the head architecture, focusing on refinement and object detection facilitation. Initially, up sampling boosts the resolution of feature maps, followed by concatenation of feature maps from the backbone network's P4 stage. Further processing involves convolutional layers and a ResBlock CBAM layer, integrating residual connections and attention mechanisms to enhance feature representation. This layer selectively emphasizes vital features and spatial regions while suppressing irrelevant ones. After another round of up sampling and concatenation

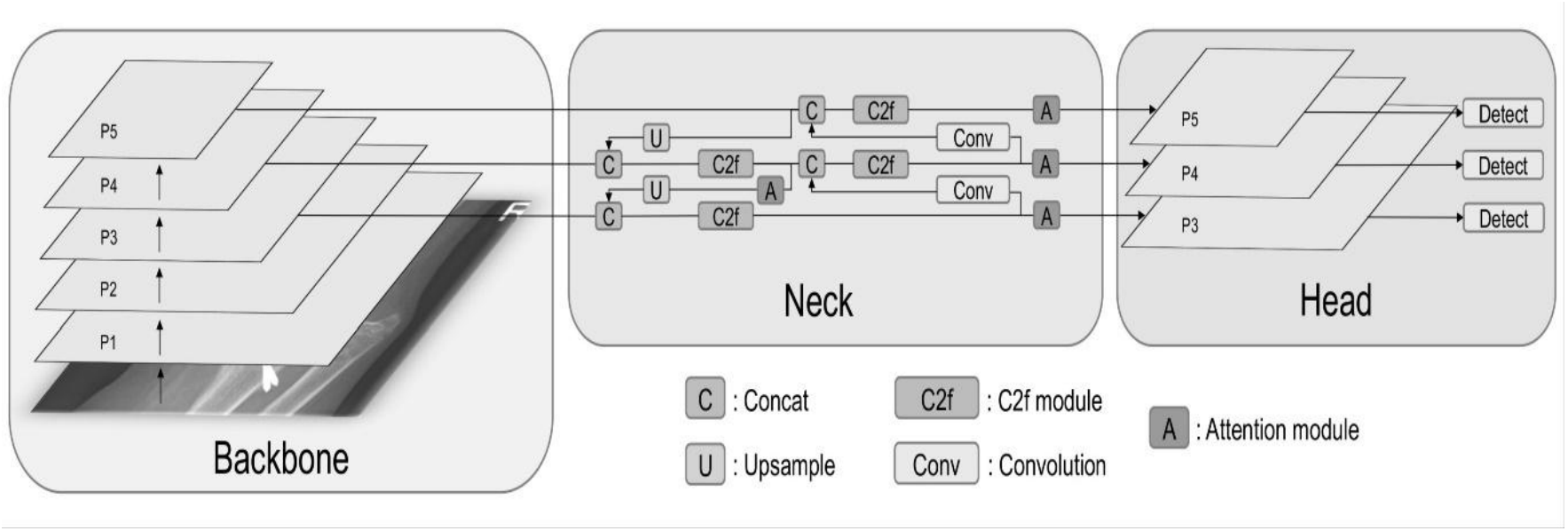

Fig. 5. Architecture of YOLO with attention mechanism

with P3 stage feature maps, another ResBlock CBAM layer refines features, capturing intricate details and spatial dependencies effectively. Subsequent stages include similar operations, culminating in the final Detect module, which aggregates feature maps from multiple stages to perform object detection, generating bounding box predictions and class probabilities. Overall, the inclusion of ResBlock CBAM layers in the head section significantly boosts the model's ability to extract meaningful features, improving object detection accuracy.

3.5 Model Deployment

The deployment of the model is done using "TensorFlow lite". Model deployment with TensorFlow involves converting trained models into a format suitable for inference on production systems. This process typically includes converting the model to a serialized format such as TensorFlow Saved Model for deployment on various platforms and devices. Once deployed, the model can perform real-time classification, enabling integration into mobile applications for inference tasks. The hardware prototype for the tomato ripeness detection system was a methodical process. Camera is used for image capture and a servo motor is used for door control due to their compatibility with computer vision algorithms. Following this, the circuitry was meticulously designed, with a focus on establishing robust connections and adhering to the specifications of each component. This phase involved creating a comprehensive schematic.

4. IMPLEMENTATION

In implementation the focus is on the practical execution of the proposed solution and the subsequent findings. Delving into the implementation process, it elucidates how the proposed solution was put into action, detailing the steps taken to realize its objectives. From the initial design to the final deployment, each stage of implementation is meticulously explored, providing insights into the practical aspects of translating theoretical concepts into tangible outcomes. Furthermore, the chapter delves into the results obtained from the implementation phase, shedding light on the efficacy of the solution in addressing the identified problem. Through meticulous analysis and interpretation of the obtained results, the chapter offers a comprehensive understanding of the solution's impact and effectiveness.

To ensure efficient communication between the front-end and back-end components, RESTful APIs were implemented using Fast API Framework. These APIs facilitated seamless data exchange and enabled the integration of various features within the system.

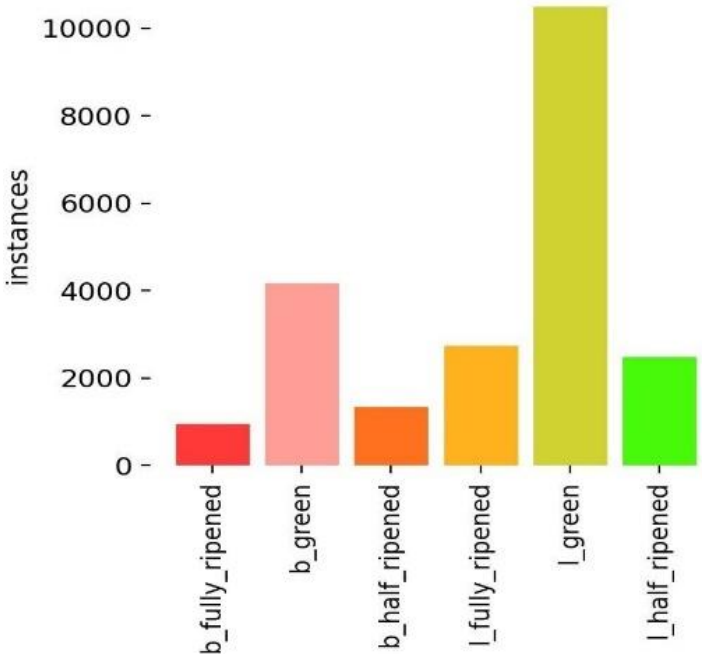

Fig. 6. Overview of the instances of the tomato in the collected data

4.1 Data Collection

For data collection, a comprehensive dataset comprising 2000 images with over 10,000 instances of various tomato varieties at different ripeness stages was curated. This diverse dataset offers a rich representation of tomatoes in real-world scenarios, enabling the model to learn robust features across different ripeness levels. By including a wide range of tomato types and ripeness stages, the dataset provides valuable insights into the variability present in tomato images, enhancing the model's ability to generalize to unseen data. y and disease identification model.

4.2 Pre Processing

In the preprocessing stage, two main steps are carried out to prepare the data for further analysis. Firstly, resizing is performed to adjust the dimensions of the input data, ensuring consistency and compatibility with the subsequent steps. Secondly, Augmentation artificially expands the dataset by applying random transformations like rotation, flipping, zooming, color jittering, and shifting. Helps prevent overfitting by training the model on a wider range of image variations, enhancing generalization.

Input: Path to the root folder containing the images.
Description: Tomato images
Output: Resized Images

Algorithm 4.1 Resizing the images

```
function RESIZE IMAGES(image list, desired width, desired height) resized
    images ← []
    for each image in image list
        do img ← READ
        IMAGE(image)
        img ← RESIZE IMAGE(img, desired width, desired height)
        resized images.append(img)
    end for
    return resized images
    end function
```

Algorithm 4.1 is to is to resize a list of input images to a desired width and height. It starts by receiving a list of images along with the desired dimensions for resizing. Then, it iterates through each image in the list, reading it from the file system, resizing it using a specified function, and appending the resized image to a new list. Finally, it returns the list of resized images. This algorithm is essential for standardizing the dimensions of images, which is often required for various computer vision tasks.

Input: Path to the root folder containing the resized images

Description: Images of the different ripeness stages of tomatoes

Output: Augmented Images

Algorithm 4.2 Augmentation of Images

```
1: function AUGMENT IMAGES(image list)
2:     image list: List of input images
3:     augmented images: List of augmented images
4:     augmented images ← []
5:     for each image in image list do
6:     img ← READ IMAGE(image)
7:     augmented img ← AUGMENT IMAGE(img)
8:     augmented images.append(augmented img)
9:     end for
10:    return augmented images
11: end function
```

Algorithm 4.2 is to to resize a list of input images to a desired width and height. It starts by receiving a list of images along with the desired dimensions for resizing. Then, it iterates through each image in the list, reading it from the file system, and apply augmentation function. This algorithm is essential for standardizing the dimensions of images, which is often required for various computer vision tasks.

4.3 Deep Learning Models

Algorithm 4.3 Train and Classify YOLOv8 with Pre-processed Images

```
Require: model, train data, val data, epochs, batch size, model path, image path
function TRAINANDCLASSIFYYOLOV8(model, train data, val data, epochs, batch size,
model path, image path)
    Step 1: Load Model
    yolo model ← LOAD MODEL("yolov8.pt)
    Step 2: Train Model
    yolo model.TRAIN(
    data=train data,
    val data=val data,
    epochs=epochs,
    batch size=batch size
    )
    Step 3: Save Trained Model
    SAVE MODEL(yolo model, model path)
    SAVE MODEL(yolo model, model path)
    Step 4: Load Trained Model
    yolo model ← LOAD MODEL(model path)
    Step 5: Read Image for Prediction 16: image ← READ IMAGE(image path) Step 6: Predict
Using Trained Model
    prediction ← yolo model.PREDICT(image)
    Return prediction
end function
```

Algorithm 4.3 It begins by loading a pre-trained YOLOv8 model via the LOAD MODEL command. This initialization step is crucial as it establishes the model's architecture and incorporates pre-existing weights, which encode knowledge learned from extensive prior training on diverse datasets. Once initialized, the function proceeds to the training phase, a iterative process characterized by a series of epochs, where each epoch represents a complete pass through the entire training dataset. During each epoch, the function partitions the training dataset into manageable batches, enabling efficient processing. For each batch, the model undergoes a sequence of operations. It first conducts a forward pass, where input images are propagated through the network, yielding predictions regarding the presence and location of objects within the images. Subsequently, the model computes the loss, a quantitative measure of the disparity between its predictions and the ground truth annotations associated with the images. This loss, typically comprising components such as localization loss, confidence loss, and classification loss, serves as a feedback signal guiding the model's learning process.

Following loss computation, the model engages in backpropagation, a fundamental mechanism for updating its parameters to minimize the computed loss. Backpropagation involves the calculation of gradients, which represent the sensitivity of the loss function to changes in the model's parameters. These gradients are subsequently utilized by an optimization algorithm, such as stochastic gradient descent or Adam, to adjust the model's weights in a direction that reduces the loss. This iterative process of forward pass, loss computation, backpropagation, and weight update continues until all batches within the epoch have been processed. After completing the designated number of epochs, the model's performance is evaluated using a separate validation dataset. This evaluation serves as a critical checkpoint, allowing the assessment of the model's generalization capabilities and its propensity for overfitting. Metrics such as mean average precision (mAP), precision, and recall are

commonly employed to quantify the model's performance during validation.

Finally, upon the conclusion of the training process, the trained model is saved to a specified model path using the SAVE MODEL command. This facilitates future utilization of the trained model for inference tasks, such as object detection in new images, including those depicting tomatoes. Additionally, the function supports the loading of saved models, enabling seamless integration into subsequent workflows or applications, thereby ensuring the efficient deployment of trained YOLOv8 models for real-world tasks.

Algorithm 4.4 Train and Classify YOLOv8 with Preprocessed Images

```
Require: model, train data, val data, epochs, batch size, model path, image path
function TRAINANDCLASSIFYYOLOV8(model, train data, val data, epochs, batch size,
model path, image path)
    Step 1: Load Model
    yolo model ← LOAD MODEL("yolov8Resblock.pt)
    Step 2: Train Model
    yolo model.TRAIN(
    data=train data,
    val data=val data,
    epochs=epochs,
    batch size=batch size )
    Step 3: Save Trained Model
    SAVE MODEL(yolo model, model path)
    Step 4: Load Trained Model
    yolo model ← LOAD MODEL(model path)
    Step 5: Read Image for Prediction 16:  image ← READ IMAGE(image path)
Step 6: Predict Using Trained Model
    prediction ← yolo model.PREDICT(image)
    Return prediction
end function
```

The algorithm 4.4 orchestrates the comprehensive process of training a YOLOv8 model integrated with a ResBlock CBAM attention mechanism and subsequently leveraging this trained model to classify tomato images. Initially, the algorithm assumes several inputs: the model architecture (model), the datasets for training (traindata) and validation (valdata), the number of training epochs (epochs), the batch size (batchsize), and the file paths for both saving and loading the model (modelpath) and for reading the tomato image to be classified (imagepath).The algorithm commences by loading the specified YOLOv8 model architecture and its parameters using the LOADMODEL function. Subsequently, it initiates the training process for the model by employing the provided training and validation datasets. During training, the model iterates over the dataset for the designated number of epochs, updating its parameters through backpropagation and optimization algorithms such as stochastic gradient descent or Adam. This iterative process refines the model's ability to accurately detect and classify objects, including tomatoes, in images. Upon completion of training, the trained model is preserved to a specified file path via the SAVEMODEL function, facilitating future accessibility and deployment. Following the training phase, the algorithm reloads the saved model from the designated file path using the LOADMODEL function. Subsequently, it reads the tomato image specified by the provided file path utilizing the READIMAGE function. Leveraging the reloaded model, the algorithm then predicts objects present in the tomato image using the PREDICT function. This prediction phase typically involves the detection of objects within the image, such as tomatoes, and the delineation of their respective bounding boxes and associated labels.

Finally, the algorithm returns the prediction results, encapsulating critical information about the detected objects within the tomato image. These results furnish users with valuable insights into the classification and localization of tomatoes within the input image, facilitating subsequent decision-making processes or downstream analyses. By meticulously delineating the training and classification procedures, the algorithm furnishes a systematic and structured approach for harnessing the power of YOLOv8 with ResBlock CBAM in tomato image analysis.

5. RESULTS AND DISCUSSION

The images collected undergo preprocessing and are subsequently trained using two distinct deep learning models: YOLOv8 and YOLOv8 with a ResBlock CBAM attention mechanism. These models are assessed using evaluation metrics including precision, recall, F1 score, and Mean Average Precision (mAP). The performance metrics are then compared, and the most effective model is chosen for deployment Mance and reliability in real-world scenarios.

5.1 Performance Metrics of Pretrained YOIO Model

The Performance metrics of Pretrained YOLO model is given below:

Precision=0.96
Recall=0.96
F1score=0.75

As a preliminary and almost obligated test of *blackOilFoam* we addressed the classical Buckley–Leverett displacement problem: a 1D two-phase incompressible case with semi-analytical solution, in which water is injected at one end displacing the oil originally present in the domain (Wang et al., 2023). Although we skip here most of the details for brevity purposes, it is important to say that the solution is characterized by a discontinuity representing the sharp front of saturation of the displacing fluid. In a numerical solution, however, the front is usually somehow diffused, making this difference a suitable candidate for comparison purposes. Our solver had no difficulties in dealing with this problem, producing results in which the front lies within a 5%

The Recall-Confidence Curve shows that the model maintains high recall at lower confidence levels across most classes, with a gradual decline as confidence increases. The Precision-Confidence Curve indicates that precision improves with higher confidence across all classes.l green shows the highest precision consistently, while l half ripened starts with lower precision and increases more slowly compared to other classes. The high precision indicates that the model makes accurate predictions, while the recall demonstrates its effectiveness in identifying positive instances. The overall performance metrics suggest that the model is reliable and effective for the given classification task.

5.2 Performance Metrics of YOLO Model with Attention Mechanism

The Performance metrics of YOLO with attention mechanism model is given below:

Precision=0.96
Recall=0.96
F1score=0.75

After this preliminary step turned successful, we were able to approach more complex cases involving the three phases the code should be able to deal with. We started by simulating Example 1 in (Zhang et al., 2018), a 1D saturated case with three-phase compressible flow, non-negligible formation compressibility, and capillary effects. Then, we engaged a 2D problem discussed in (Srivastava et al., 2014; Milczarek et al.,2009), considering gas oil compressible flow and the presence of a third irreducible water phase, with negligible rock compressibility and capillary effects. Finally, we focused on the 1st *SPE comparative case*, which consists in a 3D three-phase compressible problem with gas injection.

The model's performance, as depicted in the Recall-Confidence and Precision-Confidence curves, shows that the recall generally starts high and decreases as confidence increases, indicating that the model can correctly identify most positive instances at lower confidence levels. The precision, on the other hand, starts lower and increases with higher confidence, suggesting that while the model's predictions become more accurate with higher confidence, fewer true positives are identified. Overall, the model maintains a good balance between recall and precision, with strong performance in accurately classifying instances at higher confidence levels.

5.3 Result Visualization

The images below illustrate the disparity in tomato detection achieved by YOLOv8 and YOLOv8 with attention (ResBlock CBAM). By comparing the outputs of both models, it's evident that YOLOv8 with attention offers enhanced precision and accuracy in identifying tomatoes. This superior performance is crucial, as YOLOv8 with attention is the chosen model for deployment in our prototype, ensuring robust and reliable tomato classification. The tomatoes detected using YOLOv8 pretrained model and the YOLOv8 attention mechanism is shown in 5.3.

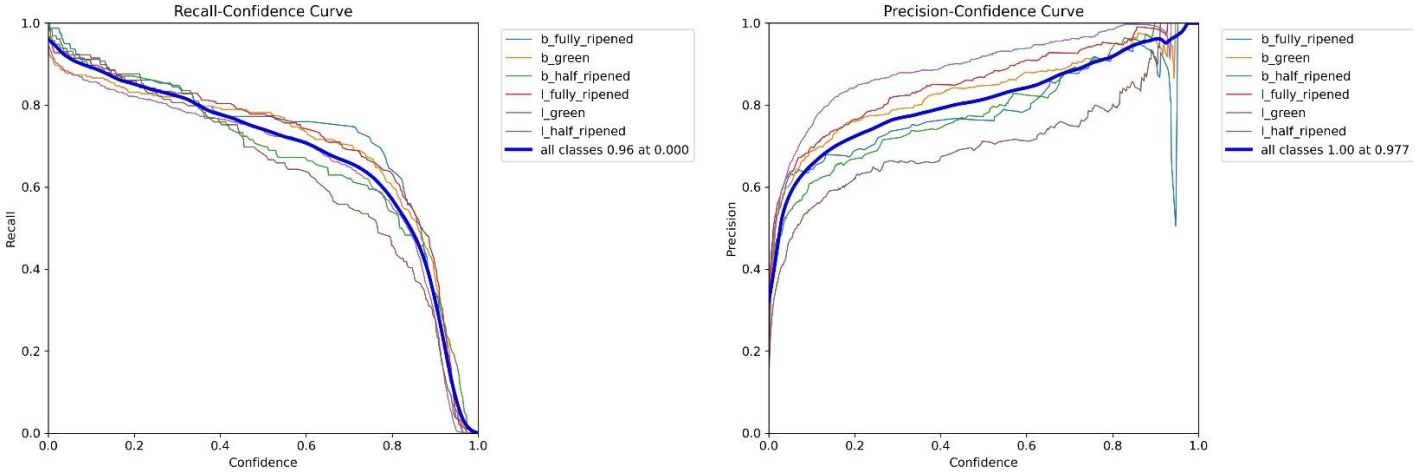

Fig. 7. Precision and Recall Chart of YOLO pretrained model

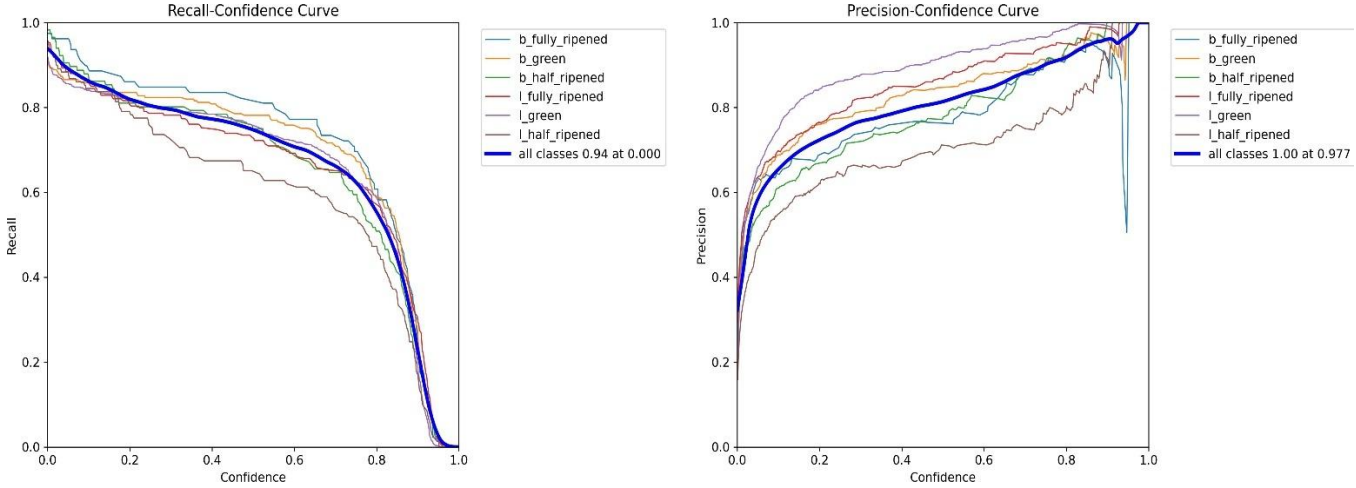

Fig. 8. Precision and recall chart of YOLO model with attention mechanism

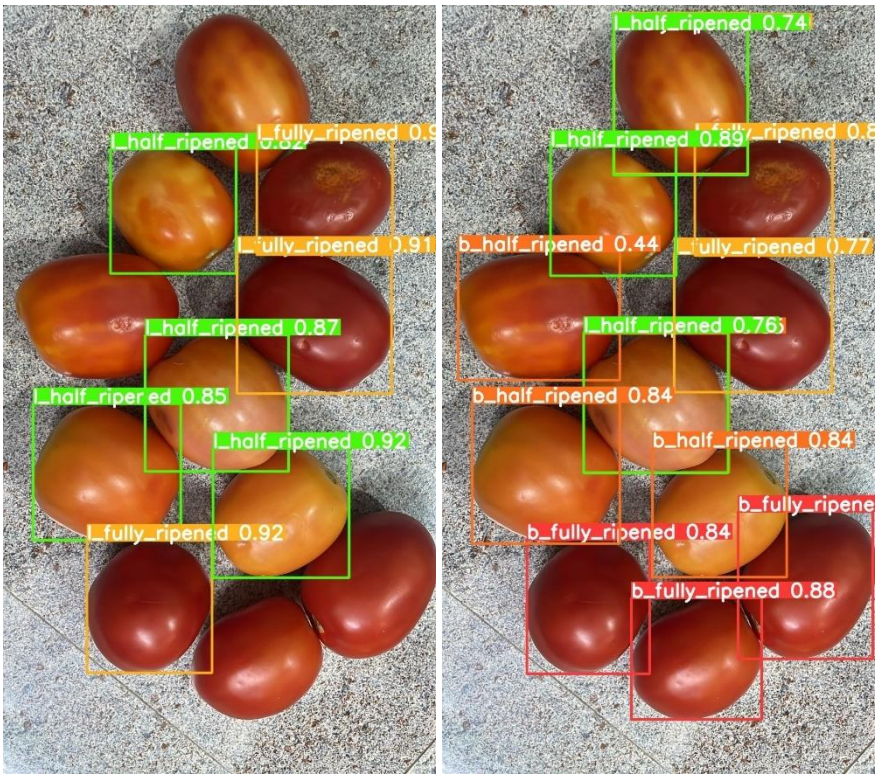

Fig. 9. Tomatoes detected using both models

Fig. 10. Tomato sorting prototype

The prototype has been tested with a sample of 150 tomatoes, and the results were highly satisfactory. Out of the 150 tomatoes, 145 were classified correctly into their respective classes, indicating a high accuracy rate of approximately 96.7%. This high level of accuracy demonstrates the effectiveness and reliability of the classification algorithm employed. This successful testing with 150 samples provides a strong foundation for further development. The results imply that the system is capable of handling larger datasets and more complex classification tasks in real-world scenarios. The robustness of the algorithm ensures that the system can be scaled up, making it suitable for industrial applications where large volumes of tomatoes need to be classified quickly and accurately (Goyal et al., 2022; Hu et al., 2016)

6. CONCLUSIONS AND FUTURE WORK

In essence, this project aimed to leverage advanced deep learning techniques, particularly utilizing the YOLOv8 model with an attention mechanism, for accurate tomato detection and classification. With the YOLOv8 model achieving a precision of 96% and YOLOv8 with attention mechanism achieving an impressive precision of 97.77%, the project showcases the effectiveness of attention mechanisms in enhancing object detection accuracy. This outcome highlights the potential of integrating attention mechanisms into deep learning models for applications like tomato classification, paving the way for more precise and reliable automated sorting processes in agricultural settings. Furthermore, the deployed prototype incorporating YOLOv8 with attention mechanism not only achieves high precision in tomato classification but also significantly reduces manual labor.By automating the sorting process, the prototype demonstrates its practical utility in decreasing manpower requirements, leading to increased efficiency and cost-effectiveness in agricultural operations.

In the future, the developed tomato classification system can serve as a robust platform for sorting a wide range of vegetables by fine-tuning image processing techniques and leveraging advanced methodologies such as transfer learning. By adapting the model to recognize and classify different vegetables, the system can be seamlessly integrated into factory production lines for automated sorting and quality control. This scalability not only enhances efficiency in vegetable processing but also opens doors to diverse applications across the agricultural and food industries.

DISCLAIMER (ARTIFICIAL INTELLIGENCE)

Author(s) hereby declare that NO generative AI technologies such as Large Language Models (ChatGPT, COPILOT, etc.) and text-to-image generators have been used during the writing or editing of this manuscript.

COMPETING INTERESTS

Authors have declared that no competing interests exist.

REFERENCES

Baltazar, A., Aranda, J. I., & González-Aguilar, G. (2008). Bayesian classification of ripening stages of tomato fruit using acoustic impact and colorimeter sensor data. *Computers and Electronics in Agriculture, 60*(2),113–121. https://doi.org/10.1016/j.compag.2007.07.005

Bishop, C. M. (2006). *Pattern recognition and machine learning*. Springer.

Bochkovskiy, A., Wang, C.-Y., & Liao, H.-Y. M. (2020). *YOLOv4: Optimal speed and accuracy of object detection*. arXiv.

Clement, J., Novas, N., Gazquez, J. A., & Manzano-Agugliaro, F. (2012). High-speed intelligent classifier of tomatoes by colour, size and weight. *Spanish Journal of Agricultural Research, 10*(2), 314–325. https://doi.org/10.5424/sjar/2012102-368-11

Deng, W., Jia, K., & Fei-Fei, L. (2009). *Imagenet: A large-scale hierarchical image database*. IEEE Conference on Computer Vision and Pattern Recognition.

Gai, R., & Yuan, H. (2023). A detection algorithm for cherry fruits based on the improved YOLO-v4 model. *Neural Computing and Applications*.

Girshick, R. (2015). *Fast R-CNN*. In *Proceedings of the IEEE International Conference on Computer Vision*.

Goyal, K., Kumar, P., & Verma, K. (2022). Food adulteration detection using artificial intelligence: A systematic review. *Archives of Computational Methods in Engineering, 29*(1), 397–426. https://doi.org/10.1007/s11831-021-09540-6.

Hu, M.-H., Dong, Q.-L., Liu, B.-L., & Luo, J.-J. (2016). Image segmentation of bananas in a crate using a multiple threshold method. *Journal of Food Process Engineering, 39*(5),427–432. https://doi.org/10.1111/jfpe.12252.

Kuznetsova, A. M., & Soloviev, V. (2020). Using YOLOv3 algorithm with pre- and post-processing for apple detection in fruit-harvesting robot. *Agronomy, 10*.

Milczarek, R. R., Faber, D., & Mazurek, W. (2009). Assessment of tomato pericarp mechanical damage using multivariate analysis of magnetic resonance images. *Postharvest Biology and Technology, 52*(2),189–195. https://doi.org/10.1016/j.postharvbio.2009.03.003

Redmon, J., & Farhadi, A. (2018). *YOLOv3: An incremental improvement*. arXiv.

Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). *You only look once: Unified, real-time object detection*. IEEE Conference on Computer Vision and Pattern Recognition.

Srivastava, S., Boyat, S., & Sadistap, S. (2014). A novel vision sensing system for tomato quality detection. *International Journal of Food Science, 2014*(1), 184894.

Wan, P., Li, X., & Zhao, Y. (2018). A methodology for fresh tomato maturity detection using computer vision. *Computers and Electronics in Agriculture, 146*, 43–50. https://doi.org/10.1016/j.compag.2017.11.019

Wang, G., Zheng, H., & Li, X. (2023). ResNeXt-SVM: A novel strawberry appearance quality identification method based on ResNeXt network and support vector machine. *Journal of Food Measurement and Characterization, 17*(5), 4345–4356.

Zhang, L., et al. (2018). Deep learning based improved classification system for designing tomato harvesting robot. *IEEE Access, 6*, 67940–67950.

Zhao, Y., Li, X., & Wang, H. (2023). Quality classification of kiwifruit under different storage conditions based on deep learning and hyperspectral imaging technology. *Journal of Food Measurement and Characterization, 17*(1), 289–305. https://doi.org/10.1007/s11694-022-01761-5

Zhou, W., et al. (2023). Tomato storage quality predicting method based on portable electronic nose system combined with WOA-SVM model. *Journal of Food Measurement and Characterization, 17*(4), 3654–3664.

Peer-review history:
The peer review history for this paper can be accessed here:
https://pr.sdiarticle5.com/review-history/144906